

DATABASE MANAGEMENT SYSTEM

MINI PROJECT REPORT

NAME: S.JAYASHREE

REG NO:2117240070129

DEPARTMENT:AI&DS

SECTION: “C”

TOPIC: courses registration and management
system

1. Abstract

The project “**Course Registration and Management System**” aims to automate the process of managing course registration in educational institutions.

It allows students to register for courses, view available subjects, and manage enrollments efficiently.

Administrators can add or remove courses, manage student records, and monitor registrations.

This system ensures **data consistency, accuracy, and easy access** using a relational database.

The project uses **MySQL** for backend database management and **SQL queries** for performing CRUD operations.

2. Objectives

- To design and implement a database for managing student course registrations.
- To maintain data consistency, integrity, and accuracy.

- To create efficient SQL queries for CRUD (Create, Read, Update, Delete) operations.
- To generate useful reports for students and administrators.

3. System Analysis

a. Problem Definition:

- In most colleges, the course registration process is still done manually using paper forms or spreadsheets
- This leads to data duplication, registration errors, and delays in report generation.

b. Existing System:

- The existing manual system lacks proper data validation and is prone to human errors.
- manage course capacities, and update student details in real time.

c. Proposed System:

- The proposed **Course Registration and Management System** overcomes these drawbacks by storing all data in a **centralized database**.
- It automates the entire registration process, ensuring **data reliability, faster processing, and easy retrieval**.

4. System Design

a. Entity Relationship (ER) Diagram:

Entities:

- Student
- Course
- Registration
- Admin

• Relationship:

- One Student can register for many Courses.
- One Course can have many Students registered
- Admin manages courses and registrations.

• Schema Diagram:

Table Name	Primary Key	Foreign Key	Description
Student	student_id	–	Stores student details

Course	course_id	–	Stores course details
Registration	reg_id	student_id, course_id	Stores course registration info
Student Admin	admin_id	–	Stores admin login details

5. Implementation:

a. Software & Hardware Requirements:

- Software: MySQL / SQLyog / phpMyAdmin
- Hardware: Minimum 4GB RAM, 1.6GHz processor
- Operating System: Windows or Linux

6. SQL QUERIES

1 Create Database:

```
CREATE DATABASE course_registration;
```

**2**

Select Database

```
USE course_registration;
```

**3**

Create Tables:

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(100),  
    department VARCHAR(50),  
    year INT  
);
```

Student Table:

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50),  
    department VARCHAR(30),  
    year INT  
);
```

b) Course Table:

```
CREATE TABLE Course (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(50),  
    credits INT  
);
```

c) Registration Table:

```
CREATE TABLE Registration (  
    reg_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT,  
    course_id INT,  
    reg_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Student(student_id),  
    FOREIGN KEY (course_id) REFERENCES Course(course_id)  
);
```



Insert Data:

a) Insert into Student Table:

```
INSERT INTO Student (student_id, student_name, department, year)  
VALUES  
(101, 'Ananya', 'CSE', 2),
```

(102, 'Ravi', 'ECE', 3),
(103, 'Meena', 'IT', 1),
(104, 'Arun', 'AI&DS', 2);

b) Insert into Course Table:

```
INSERT INTO Course (course_id, course_name, credits) VALUES  
(201, 'Database Management System', 3),  
(202, 'Operating Systems', 4),  
(203, 'Data Structures', 3),  
(204, 'Machine Learning', 4);
```

c) Insert into Registration Table:

```
INSERT INTO Registration (student_id, course_id, reg_date) VALUES  
(101, 201, '2025-10-01'),  
(101, 202, '2025-10-02'),  
(102, 203, '2025-10-03'),  
(103, 204, '2025-10-04'),  
(104, 201, '2025-10-05');
```



5 Data Retrieval Queries:

a) View All Students


```
SELECT * FROM Student;
```

b) View All Courses

```
SELECT * FROM Course;
```

c) View All Registrations

```
SELECT * FROM Registration;
```

d) Display Students with Their Registered Courses (JOIN)

```
SELECT  
  s.student_name AS Student,  
  s.department AS Department,  
  c.course_name AS Course,  
  c.credits AS Credits,  
  r.reg_date AS Registration_Date  
FROM  
  Registration r  
  JOIN Student s ON r.student_id = s.student_id  
  JOIN Course c ON r.course_id = c.course_id  
ORDER BY s.student_name;
```



Create a View

```
CREATE VIEW StudentCourseView AS
```

```
SELECT
    s.student_name,
    c.course_name,
    c.credits,
    r.reg_date
FROM Registration r
JOIN Student s ON r.student_id = s.student_id
JOIN Course c ON r.course_id = c.course_id;
```

7 Retrieve Data from the View

```
SELECT * FROM StudentCourseView;
```

8 Aggregate Query (Report)

Display total number of students per course

```
SELECT
    c.course_name,
    COUNT(r.student_id) AS total_students
FROM
    Registration r
JOIN Course c ON r.course_id = c.course_id
GROUP BY c.course_name;
```


Output:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	student_id	student_name	department	year
▶	1	Arun Kumar	CSE	2
	2	Priya Devi	ECE	3
	3	Vijay Raj	AI&DS	1
*	NULL	NULL	NULL	NULL

Student 8 x Apply Revert

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Tables_in_course_registration
▶	course
	instructor
	registration
	student
	studentcourseview

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	registration_id	student_id	course_id	registration_date
▶	2	1	101	2025-10-30
	3	2	103	2025-10-30
	4	1	101	2025-10-30
	5	1	102	2025-10-30
	6	2	103	2025-10-30
	7	1	101	2025-10-30
	8	1	102	2025-10-30
	9	2	103	2025-10-30
	10	1	101	2025-10-30
	11	1	102	2025-10-30
	12	2	103	2025-10-30

Registration 11 x Apply Revert

Result Grid
Form Editor
Field Types

registration_id	student_name	department	course_name	instructor	registration_date
10	Arun Kumar	CSE	DBMS	Dr. Ramesh	2025-10-30
7	Arun Kumar	CSE	DBMS	Dr. Ramesh	2025-10-30
4	Arun Kumar	CSE	DBMS	Dr. Ramesh	2025-10-30
21	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30
18	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30
15	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30
12	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30
9	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30
6	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30
3	Priya Devi	ECE	Data Structures	Dr. Karthik	2025-10-30

student_name	department
Arun Kumar	CSE
Arun Kumar	CSE
Arun Kumar	CSE
Arun Kumar	CSE
Arun Kumar	CSE
Arun Kumar	CSE

7. Results and Discussion:

The **Course Registration and Management System** was successfully developed and implemented using **MySQL** as the backend database.

The system efficiently handles student registration, course management, and data retrieval through SQL queries.

The results of the implemented queries show that:

- The tables were created successfully with proper relationships between students, courses, and registrations.
- Data insertion and retrieval operations were executed accurately without redundancy.
- The JOIN queries displayed meaningful relationships between students and their registered courses.
- Views and aggregate queries (like total students per course) provided clear, structured outputs for reporting purposes.

The system meets its objectives by automating the manual registration process, ensuring **data consistency, integrity, and fast access** to academic information

. It also demonstrates strong normalization and relational database design principles.

Possible future improvements could include adding:

- A user interface (UI) for students and administrators.
- Login authentication for security.
- Course scheduling and grade management modules.

8. Conclusion:

The **Course Registration and Management System** project successfully demonstrates how a database-driven system can replace a manual registration process.

Using **SQL queries and relational database concepts**, it achieves efficient storage, retrieval, and management of student and course data.

The system ensures data reliability, reduces manual effort, and supports easy record maintenance.

It highlights the importance of **Entity-Relationship design, normalization, and SQL operations** in building real-world database applications.

In the future, this project can be extended by integrating a **web interface** using technologies like **HTML, CSS, PHP, or Python Flask**, allowing students to register and view their courses online.

